MODULE 04 / TEAMMATE FIELD GUIDE

Talent Check

Compare a saved candidate's twelve explicit competency self-ratings with the selected company's twelve expected levels. Show what is known, the gaps and every part of the arithmetic.

JD | Resume | Profile | **Talent Check** | Match + GPS

The shared journey; each module also runs independently.

You receive	You return
A saved profile ID, a selected company ID, and explicit real or synthetic benchmark mode. A JD is not required.	ReadinessResult: score or null, exactly twelve category rows, completeness, evidence coverage, benchmark provenance/hash and an inspectable receipt.

Your responsibility

Participant Brief p.8 requires a company readiness score plus per-category gaps across all twelve RADIX categories. Our host update identifies staging_company_skill_levels as the read-only source. Synthetic data cannot satisfy official-data acceptance.

A successful handoff looks like this

Two different rated profiles produce explainably different results. Missing ratings or unavailable company data are shown honestly instead of silently substituting values.

> **Start from the working baseline**
>
> Assume YOU own this module. Run and understand the supplied working source, finish or improve your part, and test it. Push the verified working copy to YOUR OWN GitHub repository. Return its URL, commit SHA, actual test results and known limitations to the Scrum Master. No central merge or PR is required from you.

Before your module	After your module
Module 3 supplies the saved profile. The read-only benchmark repository supplies company expectations; it is separate from app-owned profile storage.	The user receives a company-level readiness explanation. Module 5's GPS can reuse the same pure readiness evaluator for clearly hypothetical category changes.

Read in order: 2 build plan | 3 screen layout | 4 API | 5 logic | 6 integration | 7 tests | 8 presentation | 9 exact field reference

02 / IMPLEMENTATION PLAN

What to do, in order.

Keep changes inside your feature folder. Each step has an observable result; do not begin by replacing the whole app.

01 Agree what readiness means

Read scoring.py and the formula on page 5. The equal-weight capped ratio is a team design decision, not an employer screening algorithm.

02 Load a saved profile

Use the same Profile contract as Module 3. Do not assign ratings, replace Unknown with zero or scrape named skill counts.

03 Read a valid source

Keep organizer access GET-only. Inspect actual schema before configuring mappings/filters. Each company needs exactly twelve unique valid 1-10 expectations.

04 Handle unavailable data

Return a clear source error. Synthetic lab mode must be deliberately selected and permanently labelled; never silently use it as official data.

05 Compute one row per category

Show candidate level, required level, shortfall, gap, attainment and contribution. Unknown values produce nulls; final score requires all twelve levels.

06 Expose provenance

Show source mode/table, fetched time/hash and rule version through result/receipt. Evidence coverage is a separate count, not a score multiplier.

07 Prove reuse

Compare two profiles, verify arithmetic and feed the same evaluator to GPS. Do not create a second inconsistent formula.

Files to open first

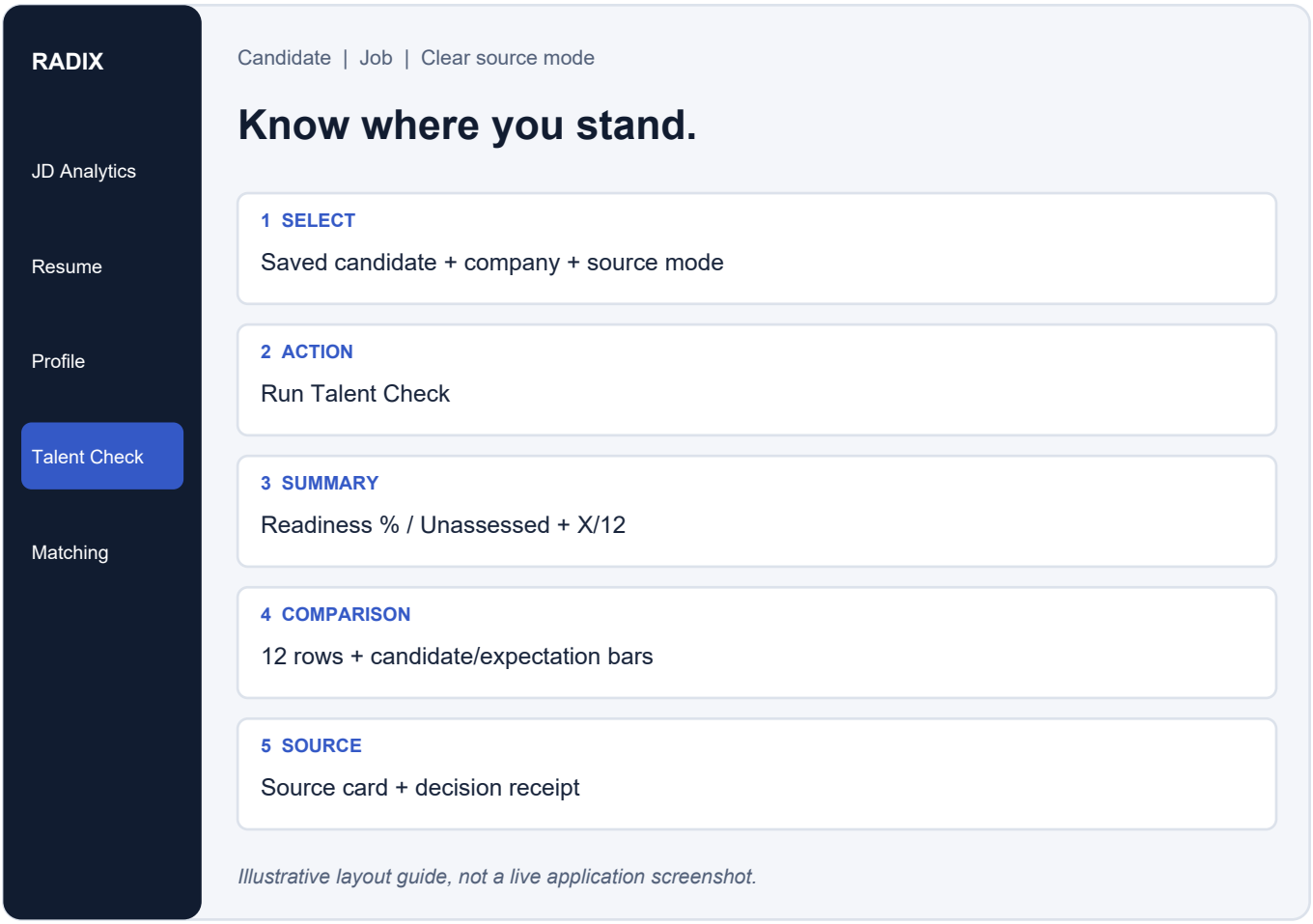
All paths below are relative to talent_check/.

File	What it owns
backend/benchmark_repository.py	Read-only source, mapping, validation, cache and permitted export loading.
backend/scoring.py	Pure twelve-category readiness arithmetic; reused by GPS.
backend/router.py	Company/status/evaluate endpoints and result receipts.
frontend/TalentCheckPage.tsx	Company comparison, source provenance, chart and row evidence.

03 / SCREEN AND USER EXPERIENCE

What it should look like.

Keep the current app shell, navigation, typography and shared controls. The drawing shows information order, not a redesign request.



Screen state	Expected visible behavior
No profile	Ask for a saved profile, not a raw CV score.
No real benchmark	Explain missing source; offer an explicit labelled lab mode.
Some ratings unknown	Show known rows and completeness; final score null, not 0%.
All twelve assessed	Show bounded score, all twelve gaps and source information.

Visual rules: 16 px body/form text, 14 px minimum supporting labels; clear headings; 44 px touch targets. Navy #111C30, blue #3459C6, white cards and #F4F6FA canvas. Use the existing Inter web font. On mobile, stack panels and keep actions readable. Do not rely on color alone for status.

04 / EXACT API BOUNDARY

Input, output and format.

Use the current Pydantic/OpenAPI contract. A UI label can change; an agreed JSON field cannot be renamed independently.

Endpoint	Body or result
GET /api/talent-check/companies	?mode=real|synthetic&refresh=false; returns IDs/names and provenance.
GET /api/talent-check/benchmark-status	Source availability/error; separate from process liveness.
POST /api/talent-check/evaluate	profile_id + company_id + benchmark_mode; returns ReadinessResult.

Complete synthetic evaluation request; replace ID with current saved ID

```
{
  "profile_id": "CURRENT_PROFILE_ID",
  "company_id": "lab-data",
  "benchmark_mode": "synthetic"
}
```

Current synthetic response excerpt

Only selected fields are shown below. Page 9 describes the complete shape. Snapshot UUIDs are teaching examples, not IDs to reuse in a running app.

```
{
  "company": "Synthetic Data Co",
  "company_id": "lab-data",
  "readiness_score": 74.9,
  "status": "complete",
  "rating_completeness": 12,
  "evidence_coverage": 5,
  "rules_version": "readiness-1.0-equal-capped"
}
```

Every mutation needs X-ProofPath-Client: web (or cli). Cloud requests also need the current user's Bearer JWT. The shared web helper supplies these headers. File upload uses multipart; never manually set its boundary. JSON fields use snake_case.

05 / REASONING AND BUSINESS RULES

Show the arithmetic, not only the percentage.

For each category, divide the candidate level by the required level and cap the result at 1. Average all twelve category attainments. Stronger skills cannot compensate beyond the cap for weaker categories.

```
attainment = min(candidate / required, 1)
shortfall = max(required - candidate, 0)
gap = candidate < required
contribution = 100 * attainment / 12
score = round(100 * sum(attainments) / 12, 1)
```

Situation	Exact expected reasoning
Simple synthetic arithmetic	If all twelve required levels are 8 and all candidate levels are 4, every attainment is 0.5; readiness = 50.0%.
Raise one of those ratings to 8	One category reaches 1.0; the remaining eleven stay 0.5. New score = 54.2%. The change is explainable.
Unknown candidate category	Return candidate_level=null, gap=null and attainment/contribution=null for that row. Final readiness_score=null, status=incomplete.
Mira synthetic SQL row	Candidate 4, invented requirement 9: attainment 4/9; shortfall 5; contribution 3.7037 points out of 100.
Evidence added	The evidence-linked count may change. The score must not change unless an explicit candidate rating or benchmark changes.

Important edge case

Real expectations must come from authorized staging_company_skill_levels reads or a permitted genuine export. Reject missing, duplicate, boolean, out-of-range or conflicting categories; do not fill gaps with guesses. The current cache is 15 minutes; refresh is explicit.

Never cross this boundary

All examples here are synthetic. No official company bar has been verified. The formula is a self-reported readiness estimate, not a hiring probability, guarantee or employer-approved cutoff.

06 / INTEGRATION AGREEMENT

Own it. Test it. Publish your part.

Your task is a working module on your own GitHub. Keep the supplied shared contracts and exact output formats so the Scrum Master can integrate the returned code.

Team rule	What you do
Service boundary	backend/scoring.py: evaluate(profile, company, bundle) -> result dictionary. The router validates ReadinessResult and records the receipt; GPS reuses evaluate.
One feature owner	Own talent_check/backend, frontend, tests and docs. Keep the common support area in your clone. Run your module independently before claiming it works.
One contract source	Read shared/contracts/models.py. OpenAPI and generated TypeScript follow it. Do not maintain a second handwritten schema or change categories alone.
One storage/auth path	Use shared store/document/auth helpers. Do not add a second database, anonymous cloud owner, public CV URL or frontend API secret.
One current record	Use actual returned IDs/revisions, owned by the signed-in user. Pass saved IDs to consumers; never hardcode teaching snapshot IDs.
Your GitHub	Create a private repository on YOUR account. Rename the old origin to scrum-master, set your new origin, review staged files and scan for secrets, then commit and push. See SUBMIT_YOUR_REPOSITORY.md.
Return working evidence	Fill DELIVERY.md with your URL, commit SHA, actual commands/results, three normal and three failure cases, synthetic screenshots and known bugs. If a check fails, fix it or report it honestly; do not copy passing counts.

The official category vocabulary

Every extracted skill has exactly one official code or OTHER. The extra descriptive skill families are filters, not replacement categories. Only the twelve official codes can have numeric ratings or company expectations.

Category	Category
COD - Coding	CLOUD - Cloud
DSA - Data Structures & Algorithms	SQL - SQL
OOD - Object-Oriented Design	SWE - Software Engineering
APTI - Aptitude	SYSD - System Design
COMM - Communication	NETW - Networking
AI - AI	OS - Operating Systems

Keep these meanings separate

Confidence = how clearly the document supports an extraction. Competency = explicit self-rating, null or 1-10. Review = a human checked the output. Verified = independently established; this app's claims remain unverified. Hypothetical = GPS preview only.

07 / RUN AND VERIFY

Your definition of done.

A module must run independently as well as inside the shared app. Record actual commands and failures; a screenshot or a copied passing count is not test evidence.

Use the existing project, not a second app

From the repository root, install the existing requirements/lockfiles if needed. Open two terminals. If the ports are already running, use them; do not kill unknown processes. Never overwrite an existing .env.

```
.\talent_check\run.ps1 api
# Separate terminal:
.\talent_check\run.ps1 web
# Real standalone API check:
.\talent_check\run.ps1 check
```

Clone the full repo and install root dependencies (README). Run these from its root. Open http://127.0.0.1:5173/; Load synthetic inputs supplies upstream fixtures. No key is needed for rules. A Groq key belongs only in the private root .env; restart Python after editing it.

Test	Pass means
All twelve known / arithmetic fixture	Exact formula and 0-100 bound; repeated inputs give identical score.
One unknown / incomplete profile	Score null; known row gaps still shown; null is not converted to zero.
Two different candidate profiles	Different appropriate category outcomes; no constant or name-based score.
Missing/duplicate/invalid benchmark	Source error; no invented expectation and no silent synthetic substitution.
Add evidence without changing rating	Readiness unchanged; evidence count remains separate.
GPS category change/reset	Same evaluator yields the projection; actual saved profile remains unchanged.

Your focused commands

```
.\.venv\Scripts\python.exe -m pytest talent_check/tests -q
npm run test -- talent_check/frontend
```

Before returning: run typecheck, lint, build and the standalone browser check. Use the project virtualenv for Python. Keep assertions meaningful and record actual results.

Error behavior to demonstrate

422 = invalid/unreadable input; 413 = size limit; 415 = unsupported file; 404 = missing/unowned record; 409 = stale revision; 503 = unavailable storage/benchmark. Groq may fall back and return a successful Extraction with llm_status=unavailable instead of an HTTP error. Show the status, not just HTTP 200.

08 / LIVE DEMO AND PRESENTATION

Show it. Explain it. Hand it over.

Suggested speaking time: four minutes per module. Five modules use about 20 minutes, leaving 10 for the problem, architecture, challenge and close within the 30-minute presentation. Q&A is separate (15 minutes).

Reproducible synthetic demo

1. Select the saved synthetic Mira profile with all twelve explicitly invented ratings set to 4.

2. Open company readiness settings; choose Synthetic lab and Synthetic Data Co (lab-data). Keep the synthetic label visible.

3. Click Run Talent Check: expected fixed-fixture score 74.9%, twelve category rows, 12/12 assessed.

4. Open SQL: self-rating 4 against synthetic requirement 9 gives 3.7037 contribution points and a shortfall of 5.

5. Explain Unknown behavior and source provenance. Hand the same formula/benchmark snapshot to GPS; official benchmark validation remains pending.

What to say and show

Time	Speaking cue
0:00-0:35	My module compares a candidate with a company's general bar across twelve categories, not one job posting.
0:35-1:25	Select synthetic company and show 74.9% plus twelve rows. Clearly label the invented standards.
1:25-2:15	Open SQL and calculate 4/9, shortfall 5 and 3.7037 contribution points.
2:15-3:10	Explain unknown ratings, source provenance and why evidence never raises proficiency automatically.
3:10-4:00	Show how changing the candidate changes the score. Transition to the separate specific-JD comparison.

Transition to the next speaker

Company readiness is one view. Module 5 now asks whether this candidate's reviewed named skills cover this particular job's requirements.

Use this language honestly

Say which parts you implemented, adapted, tested or learned. If showing the working team baseline, call it the integrated team implementation. Do not claim a live model, official benchmark or feature history that has not actually been demonstrated.

09 / FIELD REFERENCE AND QUESTIONS

Keep this beside your editor.

These are exact field names from the current Python contracts. Defaults fill omitted optional fields; partial excerpts are not safe replacements for full PATCH payloads.

Group	Exact fields / meaning
Result identity	company, company_id, profile_id, profile_revision
Score / completeness	readiness_score, status=complete|incomplete, rating_completeness, evidence_coverage
Source / explanation	skillset_gap, rules_version, benchmark_source, benchmark_hash, methodology, receipt_id
Each category row	category_code, category_name, required_level, candidate_level, gap, shortfall, status, attainment, contribution, evidence, rating_origin, recommended_action
Row states	unknown, below_expected, meets_expected, above_expected. Unknown candidate_level/gap/attainment/contribution are null.
Benchmark source	source_mode, source_table, fetched_at, content_hash, notice. Keep synthetic provenance explicit; exported real data needs validated origin.

Questions you should answer without memorizing code

Why no score with one unknown? The chosen formula averages all twelve assessed categories. Treating missing ability as zero would make an unsupported claim.

Are those official Google/Microsoft expectations? No. The demo is synthetic. Genuine authorized organizer data is still needed for real company acceptance.

Why not use an LLM for the percentage? Plain deterministic arithmetic is more transparent and repeatable. The brief describes this module as plain code.

Source of truth and current limits

Read your module's README.md, PLAN.md, CONTRACT.md and USAGE.md; shared/contracts/models.py; docs/BUILD_PLAN.md; docs/DEPLOYMENT_SETUP.md; docs/LLM_EXTRACTION_IMPLEMENTATION.md; verification/RESULTS.md; and docs/TEAM_PRESENTATION_CHECKLIST.md. Full current validated synthetic JSON snapshots are under data/synthetic/handoff/. Snapshot IDs are not live IDs.

Original Participant Brief: pp.7-10 (roles/contracts), p.11 (official test pack), p.13 (stack/done). Event Plan p.4: 30 minutes presentation + 15 Q&A. Original baseline suite (see new checkout results in verification/RESULTS.md): 134 backend, 17 frontend and 14 browser tests. Live Groq quality, cloud RLS/deployment and official organizer-pack/benchmark acceptance remain pending. All guide examples are synthetic.